```
import pandas as pd
import seaborn as sns
import urllib.request

# URL of the Iris dataset from UCI Repository
url =
"https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.d
ata"

# Column names based on dataset description
column_names = ["sepal_length", "sepal_width", "petal_length",
"petal_width", "species"]

# Load the dataset into a Pandas DataFrame
iris = pd.read_csv(url, names=column_names)

# Display first few rows
iris
```

```
     sepal_length  sepal_width  petal_length  petal_width
species
0             5.1          3.5           1.4          0.2     Iris-
setosa
1             4.9          3.0           1.4          0.2     Iris-
setosa
2             4.7          3.2           1.3          0.2     Iris-
setosa
3             4.6          3.1           1.5          0.2     Iris-
setosa
4             5.0          3.6           1.4          0.2     Iris-
setosa
..            ...          ...           ...          ...
...
145           6.7          3.0           5.2          2.3  Iris-
virginica
146           6.3          2.5           5.0          1.9  Iris-
virginica
147           6.5          3.0           5.2          2.0  Iris-
virginica
148           6.2          3.4           5.4          2.3  Iris-
virginica
149           5.9          3.0           5.1          1.8  Iris-
virginica

[150 rows x 5 columns]
```

```
iris.head()
```

```
   sepal_length  sepal_width  petal_length  petal_width      species
0           5.1          3.5           1.4          0.2  Iris-setosa
1           4.9          3.0           1.4          0.2  Iris-setosa
```

```
2           4.7          3.2           1.3          0.2  Iris-setosa
3           4.6          3.1           1.5          0.2  Iris-setosa
4           5.0          3.6           1.4          0.2  Iris-setosa
```

```
iris.tail()
```

```
     sepal_length  sepal_width  petal_length  petal_width
species
145           6.7          3.0           5.2          2.3  Iris-
virginica
146           6.3          2.5           5.0          1.9  Iris-
virginica
147           6.5          3.0           5.2          2.0  Iris-
virginica
148           6.2          3.4           5.4          2.3  Iris-
virginica
149           5.9          3.0           5.1          1.8  Iris-
virginica
```

```
iris.index
```

```
RangeIndex(start=0, stop=150, step=1)
```

```
iris.columns
```

```
Index(['sepal_length', 'sepal_width', 'petal_length', 'petal_width',
       'species'],
      dtype='object')
```

```
iris.shape
```

```
(150, 5)
```

```
iris["sepal_length"]
```

```
0      5.1
1      4.9
2      4.7
3      4.6
4      5.0
      ...
145    6.7
146    6.3
147    6.5
148    6.2
149    5.9
Name: sepal_length, Length: 150, dtype: float64
```

```
iris.iloc[4]
```

```
sepal_length            5.0
sepal_width             3.6
```

```
petal_length            1.4
petal_width             0.2
species         Iris-setosa
Name: 4, dtype: object
```

```
iris.iloc[:4,:]
```

```
   sepal_length  sepal_width  petal_length  petal_width      species
0           5.1          3.5           1.4          0.2  Iris-setosa
1           4.9          3.0           1.4          0.2  Iris-setosa
2           4.7          3.2           1.3          0.2  Iris-setosa
3           4.6          3.1           1.5          0.2  Iris-setosa
```

```
iris.loc[:4,["sepal_length","petal_length"]]
```

```
   sepal_length  petal_length
0           5.1           1.4
1           4.9           1.4
2           4.7           1.3
3           4.6           1.5
4           5.0           1.4
```

```
# any missing value across each column
iris.isnull()
```

```
     sepal_length  sepal_width  petal_length  petal_width  species
0           False        False         False        False    False
1           False        False         False        False    False
2           False        False         False        False    False
3           False        False         False        False    False
4           False        False         False        False    False
..            ...          ...           ...          ...      ...
145         False        False         False        False    False
146         False        False         False        False    False
147         False        False         False        False    False
148         False        False         False        False    False
149         False        False         False        False    False

[150 rows x 5 columns]
```

```
# any missing value across each column
iris.isnull().any()
```

```
sepal_length    False
sepal_width     False
petal_length    False
petal_width     False
species         False
dtype: bool
```

```
# count of missing values
iris.isnull().sum()
```

```
sepal_length    0
sepal_width     0
petal_length    0
petal_width     0
species         0
dtype: int64
```

```
# count row wise missing values
iris.isnull().sum(axis=1)
```

```
0      0
1      0
2      0
3      0
4      0
      ..
145    0
146    0
147    0
148    0
149    0
Length: 150, dtype: int64
```

```
# count row wise missing values
iris.isnull().sum(axis=0)
```

```
sepal_length    0
sepal_width     0
petal_length    0
petal_width     0
species         0
dtype: int64
```

```
# checking data types
iris.dtypes
```

```
sepal_length    float64
sepal_width     float64
petal_length    float64
petal_width     float64
species          object
dtype: object
```

```
iris["petal_length"] = iris["petal_length"].astype('int')
```

```
iris
```

```
     sepal_length  sepal_width  petal_length  petal_width
species
0             5.1          3.5             1          0.2     Iris-
setosa
1             4.9          3.0             1          0.2     Iris-
setosa
2             4.7          3.2             1          0.2     Iris-
setosa
3             4.6          3.1             1          0.2     Iris-
setosa
4             5.0          3.6             1          0.2     Iris-
setosa
..            ...          ...           ...          ...
...
145           6.7          3.0             5          2.3  Iris-
virginica
146           6.3          2.5             5          1.9  Iris-
virginica
147           6.5          3.0             5          2.0  Iris-
virginica
148           6.2          3.4             5          2.3  Iris-
virginica
149           5.9          3.0             5          1.8  Iris-
virginica

[150 rows x 5 columns]
```

```
iris = iris.astype("int")
```

```
---------------------------------------------------------------------------
-----
ValueError                                Traceback (most recent call
last)
Cell In[23], line 1
----> 1 iris = iris.astype("int")

File ~\anaconda3\Lib\site-packages\pandas\core\generic.py:6643, in
NDFrame.astype(self, dtype, copy, errors)
   6637     results = [
   6638         ser.astype(dtype, copy=copy, errors=errors) for _, ser
in self.items()
   6639     ]
   6641 else:
   6642     # else, only a single dtype is given
-> 6643     new_data = self._mgr.astype(dtype=dtype, copy=copy,
errors=errors)
   6644     res = self._constructor_from_mgr(new_data,
axes=new_data.axes)
   6645     return res.__finalize__(self, method="astype")
```

```
File ~\anaconda3\Lib\site-packages\pandas\core\internals\
managers.py:430, in BaseBlockManager.astype(self, dtype, copy, errors)
    427 elif using_copy_on_write():
    428     copy = False
--> 430 return self.apply(
    431     "astype",
    432     dtype=dtype,
    433     copy=copy,
    434     errors=errors,
    435     using_cow=using_copy_on_write(),
    436 )

File ~\anaconda3\Lib\site-packages\pandas\core\internals\
managers.py:363, in BaseBlockManager.apply(self, f, align_keys,
**kwargs)
    361         applied = b.apply(f, **kwargs)
    362     else:
--> 363         applied = getattr(b, f)(**kwargs)
    364     result_blocks = extend_blocks(applied, result_blocks)
    366 out = type(self).from_blocks(result_blocks, self.axes)

File ~\anaconda3\Lib\site-packages\pandas\core\internals\
blocks.py:758, in Block.astype(self, dtype, copy, errors, using_cow,
squeeze)
    755         raise ValueError("Can not squeeze with more than one
column.")
    756     values = values[0, :]  # type: ignore[call-overload]
--> 758 new_values = astype_array_safe(values, dtype, copy=copy,
errors=errors)
    760 new_values = maybe_coerce_values(new_values)
    762 refs = None

File ~\anaconda3\Lib\site-packages\pandas\core\dtypes\astype.py:237,
in astype_array_safe(values, dtype, copy, errors)
    234     dtype = dtype.numpy_dtype
    236 try:
--> 237     new_values = astype_array(values, dtype, copy=copy)
    238 except (ValueError, TypeError):
    239     # e.g. _astype_nansafe can fail on object-dtype of strings
    240     #  trying to convert to float
    241     if errors == "ignore":

File ~\anaconda3\Lib\site-packages\pandas\core\dtypes\astype.py:182,
in astype_array(values, dtype, copy)
    179     values = values.astype(dtype, copy=copy)
    181 else:
--> 182     values = _astype_nansafe(values, dtype, copy=copy)
    184 # in pandas we don't store numpy str dtypes, so convert to
object
    185 if isinstance(dtype, np.dtype) and
```

```
issubclass(values.dtype.type, str):

File ~\anaconda3\Lib\site-packages\pandas\core\dtypes\astype.py:133,
in _astype_nansafe(arr, dtype, copy, skipna)
    129     raise ValueError(msg)
    131 if copy or arr.dtype == object or dtype == object:
    132     # Explicit copy, or required since NumPy can't view from /
to object.
--> 133     return arr.astype(dtype, copy=True)
    135 return arr.astype(dtype, copy=copy)

ValueError: invalid literal for int() with base 10: 'Iris-setosa'
```

```
# data normalization
from sklearn import datasets
from sklearn import preprocessing
# Create DataFrame
df = pd.DataFrame(iris)
# Display first few rows
df.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1	0.2	Iris-setosa
1	4.9	3.0	1	0.2	Iris-setosa
2	4.7	3.2	1	0.2	Iris-setosa
3	4.6	3.1	1	0.2	Iris-setosa
4	5.0	3.6	1	0.2	Iris-setosa

```
# Separate numerical features and categorical labels
features = df.iloc[:, :-1].astype(float) # Convert numerical features
to float
labels = df.iloc[:, -1] # Select the last column (class labels)

features
```

	sepal_length	sepal_width	petal_length	petal_width
0	5.1	3.5	1.0	0.2
1	4.9	3.0	1.0	0.2
2	4.7	3.2	1.0	0.2
3	4.6	3.1	1.0	0.2
4	5.0	3.6	1.0	0.2
..	...	...	...	...
145	6.7	3.0	5.0	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.0	2.0
148	6.2	3.4	5.0	2.3
149	5.9	3.0	5.0	1.8

```
[150 rows x 4 columns]

labels
```

```
0         Iris-setosa
1         Iris-setosa
2         Iris-setosa
3         Iris-setosa
4         Iris-setosa
            ...
145    Iris-virginica
146    Iris-virginica
147    Iris-virginica
148    Iris-virginica
149    Iris-virginica
Name: species, Length: 150, dtype: object
```

```
# Apply Min-Max Scaling only on numerical columns
scaler = preprocessing.MinMaxScaler()
features_scaled = scaler.fit_transform(features)

# Create a new DataFrame with scaled features and original labels
iris_normalized = pd.DataFrame(features_scaled,
columns=column_names[:-1])
iris_normalized["species"] = labels # Add the class column back

# Display first few rows
print(iris_normalized.head())
```

```
   sepal_length  sepal_width  petal_length  petal_width      species
0      0.222222     0.625000           0.0     0.041667  Iris-setosa
1      0.166667     0.416667           0.0     0.041667  Iris-setosa
2      0.111111     0.500000           0.0     0.041667  Iris-setosa
3      0.083333     0.458333           0.0     0.041667  Iris-setosa
4      0.194444     0.666667           0.0     0.041667  Iris-setosa
```

```
# Label encoding
# observe unique values for species column
df["species"].unique()
```

```
array(['Iris-setosa', 'Iris-versicolor', 'Iris-virginica'],
dtype=object)
```

```
# define label encoder object
label_encoder = preprocessing.LabelEncoder()

# Encode labels in Species column
df["species"] = label_encoder.fit_transform(df["species"])

# observe the unique values for Species column
df["species"].unique()
```

```
array([0, 1, 2])
```

```
df
```

```
     sepal_length  sepal_width  petal_length  petal_width  species
0             5.1          3.5             1          0.2        0
1             4.9          3.0             1          0.2        0
2             4.7          3.2             1          0.2        0
3             4.6          3.1             1          0.2        0
4             5.0          3.6             1          0.2        0
..            ...          ...           ...          ...      ...
145           6.7          3.0             5          2.3        2
146           6.3          2.5             5          1.9        2
147           6.5          3.0             5          2.0        2
148           6.2          3.4             5          2.3        2
149           5.9          3.0             5          1.8        2

[150 rows x 5 columns]
```

```
from sklearn.preprocessing import OneHotEncoder

# create encoder
encoder = OneHotEncoder()

# fit and transform the class column
encoded = encoder.fit_transform(iris[['species']])

# convert to array
encoded_array = encoded.toarray()

print(encoded_array)
```

```
[[1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
```

```
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
```

```
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
```

```
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]]

encoded_df =
pd.DataFrame(encoded_array,columns=encoder.get_feature_names_out(['spe
cies']))
print(encoded_df.head())

   species_Iris-setosa  species_Iris-versicolor  species_Iris-
virginica
0                  1.0                      0.0
0.0
1                  1.0                      0.0
0.0
2                  1.0                      0.0
0.0
3                  1.0                      0.0
0.0
4                  1.0                      0.0
0.0
```

```
# Apply one-hot encoding on the species column
iris = pd.get_dummies(iris, columns=['species'])

# Display first rows
print(iris.head())
```

```
   sepal_length  sepal_width  petal_length  petal_width  species_Iris-
setosa  \
0           5.1          3.5             1          0.2
True
1           4.9          3.0             1          0.2
True
2           4.7          3.2             1          0.2
True
3           4.6          3.1             1          0.2
True
4           5.0          3.6             1          0.2
True

   species_Iris-versicolor  species_Iris-virginica
0                    False                   False
1                    False                   False
2                    False                   False
3                    False                   False
4                    False                   False
```

```
# Suppose original labels are in a separate variable
# Add it back
iris['species'] = labels

# Now create numeric dummies
iris = pd.get_dummies(iris, columns=['species'], dtype=int)

iris
```

```
     sepal_length  sepal_width  petal_length  petal_width  \
0             5.1          3.5             1          0.2
1             4.9          3.0             1          0.2
2             4.7          3.2             1          0.2
3             4.6          3.1             1          0.2
4             5.0          3.6             1          0.2
..            ...          ...           ...          ...
145           6.7          3.0             5          2.3
146           6.3          2.5             5          1.9
147           6.5          3.0             5          2.0
148           6.2          3.4             5          2.3
149           5.9          3.0             5          1.8

     species_Iris-setosa  species_Iris-versicolor  species_Iris-
virginica  \
0                   True                    False
```

```
False
1                    True                      False
False
2                    True                      False
False
3                    True                      False
False
4                    True                      False
False
..                    ...                        ...
...
145                 False                      False
True
146                 False                      False
True
147                 False                      False
True
148                 False                      False
True
149                 False                      False
True

     species_Iris-setosa  species_Iris-versicolor  species_Iris-
virginica
0                      1                        0
0
1                      1                        0
0
2                      1                        0
0
3                      1                        0
0
4                      1                        0
0
..                   ...                      ...
...
145                    0                        0
1
146                    0                        0
1
147                    0                        0
1
148                    0                        0
1
149                    0                        0
1

[150 rows x 10 columns]
```

```
iris = pd.get_dummies(iris, columns=iris['species'], dtype=int)
```

```
---------------------------------------------------------------------
-----
KeyError                                  Traceback (most recent call
last)
File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:3805,
in Index.get_loc(self, key)
   3804 try:
-> 3805     return self._engine.get_loc(casted_key)
   3806 except KeyError as err:

File index.pyx:167, in pandas._libs.index.IndexEngine.get_loc()

File index.pyx:191, in pandas._libs.index.IndexEngine.get_loc()

File index.pyx:234, in
pandas._libs.index.IndexEngine._get_loc_duplicates()

File index.pyx:242, in
pandas._libs.index.IndexEngine._maybe_get_bool_indexer()

File index.pyx:134, in pandas._libs.index._unpack_bool_indexer()

KeyError: 'species'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call
last)
Cell In[45], line 1
----> 1 iris = pd.get_dummies(iris, columns=iris['species'],
dtype=int)

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:4102, in
DataFrame.__getitem__(self, key)
   4100 if self.columns.nlevels > 1:
   4101     return self._getitem_multilevel(key)
-> 4102 indexer = self.columns.get_loc(key)
   4103 if is_integer(indexer):
   4104     indexer = [indexer]

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:3812,
in Index.get_loc(self, key)
   3807     if isinstance(casted_key, slice) or (
   3808         isinstance(casted_key, abc.Iterable)
   3809         and any(isinstance(x, slice) for x in casted_key)
   3810     ):
   3811         raise InvalidIndexError(key)
-> 3812     raise KeyError(key) from err
   3813 except TypeError:
   3814     # If we have a listlike key, _check_indexing_error will
raise
```

```
   3815     #  InvalidIndexError. Otherwise we fall through and re-
raise
   3816     #  the TypeError.
   3817     self._check_indexing_error(key)

KeyError: 'species'
```

iris

```
     sepal_length  sepal_width  petal_length  petal_width  \
0             5.1          3.5             1          0.2
1             4.9          3.0             1          0.2
2             4.7          3.2             1          0.2
3             4.6          3.1             1          0.2
4             5.0          3.6             1          0.2
..            ...          ...           ...          ...
145           6.7          3.0             5          2.3
146           6.3          2.5             5          1.9
147           6.5          3.0             5          2.0
148           6.2          3.4             5          2.3
149           5.9          3.0             5          1.8

     species_Iris-setosa  species_Iris-versicolor  species_Iris-
virginica  \
0                   True                    False
False
1                   True                    False
False
2                   True                    False
False
3                   True                    False
False
4                   True                    False
False
..                   ...                      ...
...
145                False                    False
True
146                False                    False
True
147                False                    False
True
148                False                    False
True
149                False                    False
True

     species_Iris-setosa  species_Iris-versicolor  species_Iris-
virginica
0                      1                        0
```

```
0
1                       1                       0
0
2                       1                       0
0
3                       1                       0
0
4                       1                       0
0
..                    ...                     ...
...
145                     0                       0
1
146                     0                       0
1
147                     0                       0
1
148                     0                       0
1
149                     0                       0
1

[150 rows x 10 columns]
```